



UNIVERSITÀ DEGLI STUDI DI SALERNO

Dipartimento di Informatica

Corso di Laurea Triennale in Informatica

TESI DI LAUREA

# **Q-Spire: A Quantum Code Smell Analysis Tool in VS Code with LLM-Assisted Remediation**

RELATORE

Prof. Fabio Palomba

Dott. Stefano Lambiase

Università degli Studi di Salerno

CANDIDATO

**Riccardo Alfieri**

Matricola: 0512116533

Anno Accademico 2024-2025

*Questa tesi è stata realizzata nel*

sesa<sup>lab</sup>  
SOFTWARE ENGINEERING  
SALERNO



## **Abstract**

The burgeoning field of quantum computing holds immense promise for solving complex problems intractable for classical computers, yet its rapid evolution introduces new challenges, including the emergence of quantum code smells. These are symptoms in the source code that indicate potential design or implementation issues, analogous to code smells in classical software development. Currently, the available tools for identifying these quantum code smells are limited. They often lack graphical user interfaces, offering only rudimentary detection capabilities without providing further insights or suggestions for remediation.

This thesis addresses these limitations by designing and implementing a novel Visual Studio Code extension specifically aimed at identifying quantum code smells. The primary objective is to enhance the development experience in quantum programming by providing developers with a more intuitive and comprehensive tool. The developed extension is capable of detecting all currently known quantum code smells. Furthermore, leveraging Large Language Models (LLMs), it offers potential solutions and refactoring suggestions for identified smells, moving beyond simple detection to provide actionable insights.

# Contents

|                                                               |            |
|---------------------------------------------------------------|------------|
| <b>Elenco delle Figure</b>                                    | <b>iii</b> |
| <b>Elenco delle Tabelle</b>                                   | <b>iv</b>  |
| <b>1 Introduction</b>                                         | <b>1</b>   |
| <b>2 Background &amp; State of Art</b>                        | <b>3</b>   |
| 2.1 Background and Fundamental Concepts . . . . .             | 3          |
| 2.1.1 Quantum Computing . . . . .                             | 3          |
| 2.1.2 Code Smells . . . . .                                   | 4          |
| 2.1.3 Large Language Models . . . . .                         | 5          |
| 2.2 State of Art . . . . .                                    | 6          |
| 2.2.1 Tool Limitations . . . . .                              | 6          |
| 2.2.2 Work conducted with Open-Source communities . . . . .   | 6          |
| 2.2.3 Strengths of the Work . . . . .                         | 7          |
| 2.2.4 Methodological Criticisms . . . . .                     | 8          |
| 2.2.5 Thesis Work . . . . .                                   | 9          |
| <b>3 Proposed Tool</b>                                        | <b>10</b>  |
| 3.1 Differences with Already Existing Previous Tool . . . . . | 10         |
| 3.1.1 Versatility in Configuration . . . . .                  | 11         |
| 3.1.2 Analysis of Un-Transpiled Quantum Circuits . . . . .    | 11         |
| 3.1.3 Automated Circuit Handling and Simulation . . . . .     | 11         |
| 3.1.4 Re-evaluation of the Long Circuit (LC) Smell . . . . .  | 12         |

|          |                                                                                       |           |
|----------|---------------------------------------------------------------------------------------|-----------|
| 3.1.5    | Evolution of the Non-parameterized Circuit (NC) Smell . . .                           | 12        |
| 3.1.6    | Refined Approach to the Repeated set of Operation on Circuit<br>(ROC) Smell . . . . . | 13        |
| 3.1.7    | User Interface and Usability . . . . .                                                | 13        |
| 3.2      | Tool Architecture . . . . .                                                           | 14        |
| 3.2.1    | Quantum Smells: A Class-Based Taxonomy . . . . .                                      | 14        |
| 3.2.2    | Smell Detection: Architecture . . . . .                                               | 17        |
| 3.2.3    | Smell Detection Methodologies: Dynamic Approach . . . . .                             | 18        |
| 3.2.4    | Smell Detection Methodologies: Static Approach . . . . .                              | 19        |
| 3.2.5    | Smell Explanation: An LLM-Powered Architecture . . . . .                              | 22        |
| 3.3      | Design Choices . . . . .                                                              | 24        |
| 3.3.1    | Command-Line Interface (CLI) . . . . .                                                | 24        |
| 3.3.2    | Graphical User Interface (GUI) . . . . .                                              | 24        |
| <b>4</b> | <b>Evaluation</b>                                                                     | <b>26</b> |
| 4.1      | Project Selection Criteria . . . . .                                                  | 26        |
| 4.2      | Graphs Analysis . . . . .                                                             | 28        |
| <b>5</b> | <b>Conclutions</b>                                                                    | <b>30</b> |
| 5.1      | Main Conclusions . . . . .                                                            | 30        |
| 5.2      | Future Works . . . . .                                                                | 31        |
|          | <b>Bibliography</b>                                                                   | <b>32</b> |

# List of Figures

|     |                                                                                                                                                |    |
|-----|------------------------------------------------------------------------------------------------------------------------------------------------|----|
| 3.1 | Circuit on the left considered smelly just by the previous tool; circuit on the right considered smelly by both previous and Q-Spire tool. . . | 13 |
| 3.2 | Smells Classes UML Diagram. . . . .                                                                                                            | 15 |
| 3.3 | Static Smell Detection. . . . .                                                                                                                | 22 |
| 3.4 | Dynamic Smell Detection. . . . .                                                                                                               | 22 |
| 3.5 | The tool's user interface is displayed in both light (left) and dark (right) themes, illustrating its adaptable visual presentation. . . . .   | 25 |
| 3.6 | Smell Detection Instance on the file IQCode.py using dynamic analysis.                                                                         | 25 |
| 4.1 | Smells Total Occurences . . . . .                                                                                                              | 28 |
| 4.2 | Repository Smells Percentage . . . . .                                                                                                         | 28 |
| 4.3 | Smells Boxplot . . . . .                                                                                                                       | 28 |
| 4.4 | Smells Correlation Matrix . . . . .                                                                                                            | 28 |

# List of Tables

|     |                                                                 |    |
|-----|-----------------------------------------------------------------|----|
| 3.1 | Smells information used in this thesis for the tool. . . . .    | 16 |
| 4.1 | Quantum Code Smells detected in the analyzed projects . . . . . | 27 |

# Introduction

In the fast-paced world of software development, a persistent and costly issue known as technical debt often emerges. This term refers to the shortcuts and compromises made during the development process that, while offering short-term gains, lead to long-term problems [1, 2, 3, 4]. The consequences can be severe, including reduced development speed, increased maintenance costs, and a higher risk of system failures. A common symptom of technical debt is the presence of code smells: indicators that something may be wrong with the underlying code. These smells, such as duplicated code or overly complex functions, signal deeper design issues that can hinder a project's evolution and scalability [5, 6, 7].

While technical debt is a well-known challenge in classical software, its implications become even more critical when applied to new and rapidly evolving domains. Quantum computing is one such field [8, 9, 10]. Unlike classical computers that store data in binary bits, quantum computers use qubits, which can exist in multiple states simultaneously. This fundamental difference enables them to solve certain complex problems exponentially faster. However, the unique principles of quantum mechanics, such as superposition and entanglement, demand entirely new approaches to software development. As the field grows, so does the risk of introducing technical debt, which could severely impede the progress and reliability of these cutting-edge systems [11, 12].

Addressing this challenge is the central objective of this thesis. Our goal is to provide developers with a practical tool to identify and manage technical debt specifically within quantum computing systems. Although early work by Chen et

al. [13] has pioneered the identification of quantum code smells, their approach is primarily focused on detection.

The proposed solution, Q-Spire (**Q**uantum **S**nell **P**rocessing with **I**ntelligent **R**emediation and **E**xplanation), is a novel tool designed to assist developers in identifying quantum code smells as defined by Chen et al. The tool's core functionality is a sophisticated detection engine that operates through two distinct methodologies: static analysis and dynamic analysis.

The dynamic analysis approach involves the direct execution of the quantum code under examination. By running the code, the tool can monitor its behavior and identify patterns that correspond to known code smells. This method provides an accurate representation of the code's performance in a real-world scenario.

In contrast, the static analysis method simulates the execution of the target code without running it directly. This is particularly useful for analyzing individual classes or functions. A key innovation of our static analyzer is the ability to generate a temporary, executable "main" function at runtime. This allows the tool to simulate the code's behavior and detect smells in a controlled environment, providing a flexible and efficient way to analyze isolated code snippets.

Both methodologies rely on the same underlying detection algorithms, ensuring consistency in the results.

This thesis builds upon that foundation by introducing a more comprehensive and user-friendly solution. The proposed tool features a graphical user interface (GUI) that not only detects these quantum code smells, but also leverages a Large Language Model (LLM) to provide developers with detailed explanations and actionable solutions. This innovative approach aims to make the process of identifying and resolving technical debt more intuitive and accessible, ultimately helping to ensure the robustness and maintainability of future quantum software [14, 15].

To validate the effectiveness of our tool, we have conducted a comprehensive evaluation on a curated set of quantum software projects. These projects, sourced from public GitHub repositories, include some of the same projects analyzed by Chen et al. This comparative approach will allow us to directly assess the performance of our tool against a well-established baseline, providing a robust measure of its accuracy and reliability.

# Background & State of Art

## 2.1 Background and Fundamental Concepts

### 2.1.1 Quantum Computing

Quantum computing represents a novel computational paradigm that leverages the principles of quantum mechanics to process information [16, 17]. At the core of this approach lies the qubit (quantum bit), the fundamental unit of quantum information. Unlike the classical bit, which can exist only in one of two states —0 or 1— a qubit can exist in a superposition of both states simultaneously, enabling a probabilistic and parallel representation of 0 and 1 [18]. This feature, together with quantum entanglement —a phenomenon where two qubits become linked so that the state of one directly influences the other—enables quantum computers to perform massively parallel computations. Quantum circuits are constructed by combining quantum gates, which manipulate the states of qubits to implement complex algorithms.

Thanks to these properties —superposition, entanglement, and interference [19, 20, 21]— quantum computers are potentially capable of tackling computational problems that are intractable for classical computers. This leads to the concept of quantum supremacy [22, 23, 24], which refers to the ability of a quantum processor to solve a problem significantly more efficiently than any traditional computer. Currently, quantum computing is mainly used in research and development contexts, with promising applications in areas such as the simulation of physical and chemical

systems, optimization of logistical and financial processes, quantum machine learning, and post-quantum cryptography [25, 26, 27, 28, 29]. Leading companies in the field include IBM, Google, and Microsoft, which are actively investing in hardware construction, algorithm development, and the creation of dedicated software.

Despite recent advances, quantum computing still faces significant technological limitations [30, 31]. Qubits are highly sensitive to external environments and prone to decoherence—a phenomenon that rapidly degrades quantum information. Errors occur frequently, and error-correction techniques are still under development. Moreover, device scalability remains a major challenge: current systems operate with tens or hundreds of qubits, whereas large-scale industrial applications would require millions. Nevertheless, with ongoing improvements in hardware and algorithms, the future potential of quantum computing remains vast, with the capacity to profoundly transform various scientific and economic domains [32, 9, 33, 34].

Within this context emerges quantum programming, which refers to the development of algorithms designed to run on quantum computers. Due to the hardware limitations mentioned above, a hybrid approach has gained traction—combining classical and quantum resources [35, 36]. This allows the power of qubits to be harnessed in those parts of the algorithm where they offer a genuine advantage, while delegating the remaining operations to classical systems, which are more mature and efficient. These hybrid systems are currently among the most promising directions for practical applications.

### **2.1.2 Code Smells**

Meanwhile, classical software continues to evolve, along with growing attention to code maintainability and quality. Within this domain lies the concept of code smells—indicators of potential issues in the source code, such as excessive complexity, duplication, or poor readability. Although not actual bugs, code smells can undermine software quality and hinder its long-term evolution, making it harder to test and modify [37, 38, 39, 40]. These issues often stem from suboptimal design decisions and, if left unaddressed, can lead to what is known as "software rot."

When it comes to quantum software, attention to quality becomes even more

critical. The fragility of the quantum environment and the lack of resources demand careful design aimed at minimizing errors and optimizing qubit usage [41, 42]. Writing high-quality quantum code means not only ensuring the correctness of results, but also maintaining system efficiency and adaptability over time. In a field where tools are still maturing, promoting good development practices is essential for building robust and genuinely useful software.

### **2.1.3 Large Language Models**

The field of Artificial Intelligence (AI) has undergone a significant transformation in recent years, largely driven by advancements in machine learning, particularly deep learning [43, 44]. At its core, AI aims to create systems that can perform tasks requiring human-like intelligence, such as reasoning, learning, and problem-solving. A pivotal development within this field is the rise of Large Language Models (LLMs). These are sophisticated AI models trained on vast datasets of text and code, enabling them to understand, generate, and interact with human language in a coherent and context-aware manner [45, 46]. A key capability that distinguishes modern LLMs is in-context learning. Unlike traditional machine learning models that require a separate fine-tuning process for each new task, LLMs can learn from a few examples provided directly within the input prompt. This method allows the model to adapt its behavior to a specific task without altering its internal parameters [47]. LLMs are proving to be powerful tools for both learning and academic research. In a learning context, they can act as interactive tutors, providing explanations on complex topics, summarizing lengthy articles, and generating practice questions. This facilitates a more personalized and accessible educational experience [48]. Despite their remarkable capabilities, LLMs have several inherent limitations stemming from their nature and training methodology. They lack a true understanding of the world, relying instead on statistical patterns to predict the next sequence of words. Consequently, their outputs are prone to various types of errors - such as hallucination, bias and many more [49, 50].

## 2.2 State of Art

In the context of quantum software quality, a significant contribution was made by the QSmell tool, developed by Chen et al. [13] in 2023. This tool was designed to detect code smells specific to programs written using the Qiskit framework, which is one of the most widely used libraries for quantum development in Python [51, 52].

### 2.2.1 Tool Limitations

The tool, which can be invoked from the command line, requires the user to specify the smell to be analyzed, the source file to be examined, and the output file for the results. However, to function correctly, QSmell needs an outdated version of the Qiskit framework (0.37.0) and Python 3.7.8, making it effectively incompatible with more recent versions of the development environment.

Due to these constraints, even by following the official instructions provided in the project’s GitHub repository, running the tool can be problematic. Errors have been encountered related to the definition and invocation of internal functions, requiring modifications to the source code to ensure its proper functioning.

An additional limitation lies in its usability: analysis can only be performed via the terminal, requiring the user to specify the necessary parameters (smell type, input file, and output file) each time, without any graphical interface to guide the user.

Finally, and most importantly, QSmell is limited to detecting smells only, without providing specific lines to identify the smell, or explanations and suggestions for their removal. This represents a significant limitation, especially in an educational or professional context where understanding and solving problems are crucial [53, 54].

### 2.2.2 Work conducted with Open-Source communities

To validate its effectiveness, the authors conducted two distinct phases of analysis based on open-source projects: an initial evaluation aimed at verifying the tool’s accuracy and a more extensive empirical study focused on observing the prevalence of smells in real-world quantum projects.

In the validation phase, two open-source projects —qsvc and fae— were randomly

selected from a previously collected set of programs. These two projects were manually examined by two independent raters, who, based on the formal definitions and thresholds associated with each smell, manually identified the presence of smells in the code. The results obtained by QSmell were then compared with those of the raters, revealing a perfect match in terms of precision, recall, and F1-score (all equal to 100% ).

Subsequently, the empirical study focused on three widely used open-source projects (qiskit-machine-learning, qiskit-terra, and qiskit-nature), each serving as an umbrella project for multiple sub-projects (Quantum Projects, QPs). In total, 15 QPs were analyzed, excluding the two already used in the previous phase. For each of them, smells were identified using thresholds previously defined by the authors or, in the absence of pre-existing references, calculated as the median of the metrics observed across the entire sample.

### 2.2.3 Strengths of the Work

One of the most commendable aspects of the work by Chen et al. is the explicit recognition of the lack of public datasets related to quantum code smells. This gap led the authors to manually construct their own dataset, thereby demonstrating the still pioneering nature of the field and the need for ad-hoc methodologies.

The independent manual validation carried out by two raters (not involved in the tool's development) represents a further strength, as it reduces the risk of bias in the evaluation process. Moreover, the perfect match between the tool's outputs and the manually assigned labels suggests that QSmell is indeed capable of correctly detecting the smells, at least with respect to the formal definitions used.

Another positive aspect is the use of real open-source projects, which gives greater practical relevance to the study's results, albeit with some reservations (as discussed later). Finally, methodological transparency is guaranteed by the explicit definition of detection thresholds for each smell, an element that facilitates the reproducibility of the experiment.

### 2.2.4 Methodological Criticisms

Despite its interesting contributions, the work presents several methodological limitations that must be carefully considered. First, the validation phase was conducted on an extremely small sample, consisting of only two projects. This limited size makes the performance evaluation of the tool statistically weak and prevents drawing robust conclusions about its general effectiveness.

The empirical study, although extended to a total of 15 Quantum Programs (QPs), is also limited in the variety of projects analyzed, as they all come from the same ecosystem (Qiskit). The fact that they are organized in umbrella projects does not automatically guarantee the diversity required for reliable generalization of the results.

Further doubts arise regarding the definition of the thresholds used to classify projects as "smelly." In the absence of theoretical benchmarks, the adoption of the median as a threshold represents a reasonable heuristic choice but is not universally valid. Different projects could, in fact, present very diverse structural characteristics, making a single threshold value inadequate.

Additionally, the work does not propose any correlation analysis between the presence of smells and real software quality issues, such as bugs, maintenance difficulties, or performance drops. Consequently, there is a lack of "impact validation," meaning a demonstration that the identified smells actually have negative consequences on the code.

Finally, although the tool is one of the first of its kind, the authors do not propose any comparison with alternative approaches, not even by adapting traditional smells to the quantum context. This makes it difficult to assess QSmell's true positioning relative to other existing tools or techniques.

### 2.2.5 Thesis Work

Based on the limitations listed above, this thesis aims to propose a complementary approach to overcome some of the critical issues that have emerged in the current literature on quantum code smell detection. Specifically, the goal is to design and develop a tool, built on Qiskit, that can not only identify specific smells in quantum code but also provide a contextualized explanation and a resolution suggestion for each of them.

To address the usability issues of current tools, the system is designed to be intuitive and easily integrable into a quantum developer’s workflow, providing a user-friendly Graphical User Interface (GUI) to simplify the entire process.

This work, therefore, intends to not only detect code anomalies but also to actively contribute to the improvement of quantum software quality by offering informative and practical support to developers. In doing so, it aims to bridge the gap between passive smell detection and the concrete adoption of corrective practices within the development community.

## Proposed Tool

Q-Spire is a novel software tool designed for the static analysis and identification of quantum code smells within quantum circuits and programs written in Qiskit. This tool leverages a predefined set of code smell definitions, some of which have been adapted and refined for the specific challenges of quantum computing, to perform a comprehensive detection process.

The core functionality of Q-Spire is its ability to not only pinpoint these code smells but also to provide a user-friendly and accessible experience. It features an intuitive Graphical User Interface (GUI) that simplifies the analysis workflow for quantum developers and researchers.

In addition to its primary detection capabilities, the tool incorporates an advanced Large Language Model (LLM)-based methodology. This integration allows users to request detailed explanations and insights for specific detected smells, offering a richer understanding of the underlying issues and providing guidance on potential solutions and best practices.

### 3.1 Differences with Already Existing Previous Tool

While drawing inspiration from previous work, particularly the tool developed by Chen et al. (2021), Q-Spire introduces several key distinctions to enhance its functionality, user-friendliness, and relevance in the evolving landscape of quantum computing. These differences are primarily rooted in design choices related to configuration, circuit analysis, and specific smell detection metrics.

### 3.1.1 Versatility in Configuration

A notable divergence from the previous tool lies in the approach to configuration settings for smell detection. The tool by Chen et al. employs fixed, systematic metrics for each code smell. While effective, this can limit flexibility. In contrast, the proposed tool introduces a configurable system, allowing users to fine-tune specific parameters for the detection of each individual code smell. This design choice offers significant advantages for developers, enabling them to customize the tool's sensitivity based on specific project requirements, hardware constraints, or coding standards. This adaptability is crucial for researchers and practitioners working on diverse quantum applications, as it allows for a more tailored and context-aware analysis.

### 3.1.2 Analysis of Un-Transpiled Quantum Circuits

A fundamental difference from the previous tool is the decision to analyze un-transpiled quantum circuits. This approach ensures the analysis remains close to the original source code and is independent of specific hardware backends. By working on un-transpiled circuits, the tool provides consistent and reproducible results, focusing on the logical design of the circuit rather than the low-level physical implementation details introduced by the transpilation process. This allows for a more straightforward analysis of design-level code smells, which are the core issues addressed by the tool.

### 3.1.3 Automated Circuit Handling and Simulation

A key difference between the proposed tool and its predecessor lies in the method of handling quantum circuits for analysis. The tool by Chen et al. required a manual "wrapper" for each circuit to provide initial values for the analysis. While this approach provided a controlled environment, it introduced a significant manual overhead. Developers were required to create and maintain a custom wrapper for every new circuit, a process that was time-consuming and prone to human error. Furthermore, manually assigning initial values could introduce a potential for bias in the analysis, as the results would be dependent on the specific chosen values.

Q-Spire, in contrast, automates this entire process. It independently simulates the quantum circuit to be analyzed, generating the necessary initial conditions on its own. This eliminates the need for any manual wrappers, significantly reducing the effort required from the user. This automated approach ensures that the analysis is consistent, reproducible, and unbiased, as it is not influenced by a developer’s specific choices. This design choice not only improves the tool’s usability but also enhances the reliability of its results, making it more suitable for a wide range of unknown or complex quantum programs.

### **3.1.4 Re-evaluation of the Long Circuit (LC) Smell**

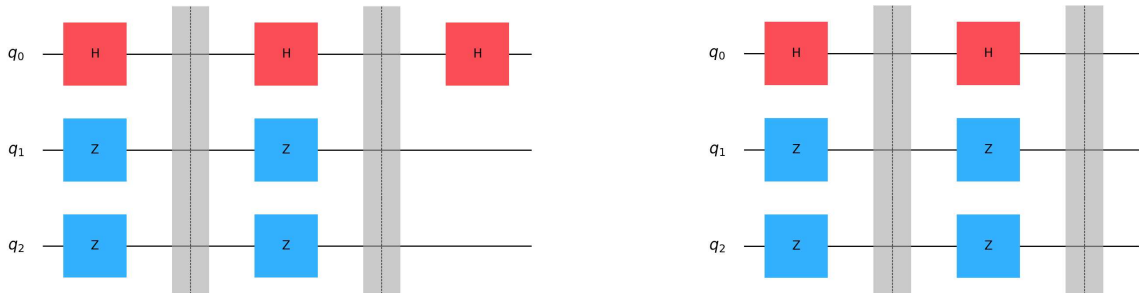
The detection of the Long Circuit (LC) smell has been refined in the proposed tool. The previous tool’s paper defined the LC smell using a metric related to the maximum gate error of a backend. However, its implementation simplified this to a product of the maximum number of parallel operations and the maximum number of operations on a single qubit. Q-Spire takes a more dynamic approach: it first checks for the presence of a target backend in the user’s code. If a backend is specified, the tool fetches its maximum gate error value and incorporates it into the calculation. If no backend is found, a configurable default value is used. The smell is then detected if the calculated value falls below a predefined threshold, providing a more context-aware and accurate assessment of circuit longevity.

### **3.1.5 Evolution of the Non-parameterized Circuit (NC) Smell**

The detection of the Non-parameterized Circuit (NC) smell has been updated to reflect the evolution of the Qiskit library. The previous tool identified this smell by counting calls to `run`, `execute`, and `bind_parameters`. Due to recent changes in Qiskit versions, the `bind_parameters` function has been deprecated and replaced by `assign_parameters`. The proposed tool accounts for this change by detecting both `bind_parameters` and `assign_parameters`. This ensures the tool remains effective across different versions of Qiskit, accommodating both legacy code and the latest releases, thereby improving its longevity and compatibility.

### 3.1.6 Refined Approach to the Repeated set of Operation on Circuit (ROC) Smell

The detection of the Repeated set of Operation on Circuit (ROC) smell has been fundamentally re-engineered. The previous tool analyzed sequential repetitions of parallel operations. Q-Spire, however, takes a different approach by analyzing the entire quantum circuit for a more meaningful type of redundancy. The circuit is logically segmented into batches of operations, where each batch contains an equal number of operations. The tool then checks if all of these batches are identical. This method is made possible by generating batches for every divisor of the total number of operations in the circuit. If such a pattern of identical batches is found, the ROC smell is identified. This approach not only provides a more comprehensive analysis of circuit redundancy but also directly points to a potential solution, namely, the use of a loop or repeat structure, which is a significant improvement over previous method.



**Figure 3.1:** Circuit on the left considered smelly just by the previous tool; circuit on the right considered smelly by both previous and Q-Spire tool.

### 3.1.7 User Interface and Usability

Beyond the core functionality, a significant distinction between the two tools lies in their user interface and overall usability. The previous tool by Chen et al. was designed as a command-line interface (CLI). While this can be powerful for experienced users, it presents a steep learning curve and can be challenging for developers less familiar with terminal environments. In contrast, the proposed tool features a graphical user interface (GUI). This choice dramatically improves

accessibility and user-friendliness, allowing users to interact with the tool intuitively, easily configure parameters, and visualize results without extensive knowledge of specific commands. This makes the tool more approachable for a wider audience, from students to quantum software engineers, thereby increasing its practical utility.

## 3.2 Tool Architecture

The architecture of Q-Spire is founded on the principles of Object-Oriented Programming (OOP), a design paradigm that allows us to model our system in a modular and extensible manner. This approach is centered on managing the relationships between parent and child classes to efficiently organize the codebase and its functionalities.

For the core Detector and Explainer modules, we specifically implemented the Factory design pattern. This creational pattern was chosen to abstract the object creation process from the client logic, ensuring the system remains flexible and easy to maintain. This deliberate application of the Factory pattern offers significant architectural advantages, enhancing the tool's quality and scalability. Primarily, it promotes modularity by separating the logic for each smell into its own dedicated module. This not only keeps the codebase clean and organized but also makes it highly extensible. The system can easily accommodate the addition of new quantum code smells without requiring any modification to the existing core classes.

### 3.2.1 Quantum Smells: A Class-Based Taxonomy

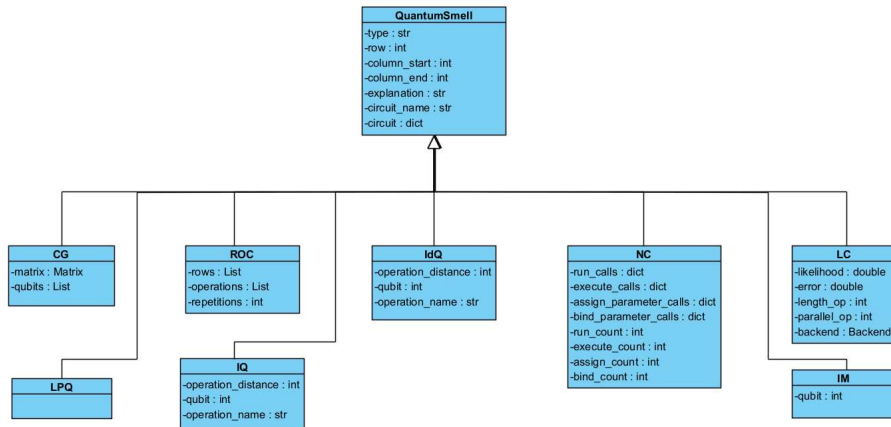
To ensure the modularity and extensibility of the proposed tool, the identified quantum smells are organized into a hierarchical class structure. This design, based on the principles of object-oriented programming, establishes a clear and systematic framework for representing and managing different types of software quality issues within quantum circuits.

### The Parent Class: QuantumSmell

The foundation of this taxonomy is the abstract class `QuantumSmell`, which serves as the parent class for all specific smell implementations. This class encapsulates the fundamental attributes and behaviors common to any detected code smell, thereby preventing redundancy and promoting code reusability. The primary attributes of this class are designed to provide essential information for smell identification and reporting, including the smell location, such as the line and column numbers within the analyzed source code, which is critical for providing actionable feedback to the developer.

### Subclasses: The Specific Implementations of The Smelly Eight

The eight identified quantum code smells are implemented as individual subclasses that inherit from the `QuantumSmell` parent class. This specialized design allows each subclass to extend the common attributes with properties specific to the nature of that particular smell. This architectural choice not only provides a robust framework for managing the current set of smells but also ensures the tool's adaptability. Future extensions, such as the inclusion of new smell types, can be seamlessly integrated by creating new subclasses that extend the base class.



**Figure 3.2:** Smells Classes UML Diagram.

The main information for each specific quantum code smell, regarding its description and detection metric, is detailed in Table 3.1, providing a centralized and organized reference for the analysis conducted by the tool.

| Acronym | Name                                                  | Description                                                                                                                                                                                                    | Detection Metric                                                                                                                                                                                                                                                                         |
|---------|-------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| CG      | Use of Customized Gates                               | Customized gates can be decomposed into built-in operators, but this process typically results in a significantly higher number of operations compared to using only built-in gates.                           | Occurrence of the UnitaryGate, HamiltonianGate or or SingleQubitUnitary gates invoked with a matrix as input.                                                                                                                                                                            |
| ROC     | Repeated set of Operations on Circuit                 | Due to hardware limits, quantum circuits should minimize repeated sequential operations to stay within execution bounds.                                                                                       | Repeated sets of operation in the whole circuit.                                                                                                                                                                                                                                         |
| NC      | Non-parameterized Circuit                             | Quantum devices share resources, so circuits should be designed parametrically to reduce communication and allow multiple inputs at once.                                                                      | Number of calls to run and execute are more than calls to bind_parameters and assign_parameters.                                                                                                                                                                                         |
| LC      | Long Circuit                                          | Unitary gates and measurements are susceptible to quantum noise, making deeper or wider circuits more likely to deviate from their intended behavior.                                                          | Likelihood of a circuit not having any error as $(1 - error)^{l \cdot c}$ , where error is the maximum error of any gate of the backend that runs the circuit, l is the the maximum number of operations in any qubit and c is the maximum number of parallel operations in the circuit. |
| IM      | Intermediate Measurements                             | Measurements disturb the quantum system and can introduce additional errors. To minimize error propagation, they should be deferred until the final step of the circuit.                                       | Non-terminal measurements.                                                                                                                                                                                                                                                               |
| IdQ     | Idle Qubit                                            | With current technology, quantum states remain stable only for brief periods. Leaving qubits idle for too long increases the risk of information loss, potentially compromising the circuit's output.          | Maximum number of operations on any qubit between the first and second operation on a specific qubit.                                                                                                                                                                                    |
| IQ      | Initialization of Qubits differently from $ 0\rangle$ | Maintaining coherence in an excited quantum state is technologically challenging. Therefore, it's advisable to keep qubits in their ground state $ 0\rangle$ for as long as possible during circuit execution. | Maximum number of operations on any qubit between the operations on any qubit.                                                                                                                                                                                                           |
| LPQ     | No-alignment between the Logical and Physical Qubits  | The physical topology of qubits influences circuit behavior, meaning results can vary depending on qubit placement. Misalignment between logical and physical qubits may reduce accuracy.                      | Call to the transpile method with no initial layout.                                                                                                                                                                                                                                     |

**Table 3.1:** Smells information used in this thesis for the tool.

### 3.2.2 Smell Detection: Architecture

The core of the tool’s functionality lies in its ability to systematically detect quantum code smells. This process is orchestrated by a set of specialized detectors, each designed to identify a specific type of smell. The architecture for these detectors is based on a modular, hierarchical class structure, which ensures scalability and facilitates the integration of new detection logic as the field of quantum software engineering evolves.

The architectural design of the detection module is built upon the Factory design pattern, centered around a core, abstract class, `Detector`. This class provides an interface and common functionalities that all specific detection processes inherit. Individual smell detectors are implemented as distinct subclasses that extend `Detector`, with each subclass encapsulating the specific logic required to identify a particular quantum code smell.

The detection process itself employs a dual-strategy approach to ensure comprehensive analysis:

#### Source Code Analysis

This method examines the structure and content of the quantum program’s source code without executing it. It is effective for identifying smells that are evident from the code’s syntax and structure, such as redundant gate chains or unnecessary qubit allocations. This approach is computationally lightweight and provides a rapid initial assessment.

#### Execution-Based Analysis

For more complex smells that depend on the circuit’s runtime behavior or state, an analysis is performed by running the code. This involves simulating the quantum circuit on a local machine to gather essential runtime information, such as the circuit’s depth, entanglement patterns, and operational flow. By examining the circuit’s state and behavior during execution, the tool can uncover issues that are not apparent from a simple code scan.

This combined methodology ensures that the tool can identify a broad range of quantum code smells, from straightforward structural issues to more subtle problems that manifest only during execution.

### 3.2.3 Smell Detection Methodologies: Dynamic Approach

The tool employs a Dynamic methodology to detect quantum code smells. This approach is predicated on the direct execution of the user-provided quantum program, as many significant quality issues only become apparent when the circuit is constructed and its behavior can be observed at runtime. This dynamic inspection is particularly effective for identifying complex smells related to the circuit's runtime behavior. To guarantee a safe and controlled analysis environment, several preliminary and runtime checks have been implemented.

#### Pre-execution Checks

Prior to execution, the tool performs a series of static analyses to mitigate potential risks associated with running arbitrary code. A primary safeguard is the detection of functions that could lead to unpredictable or malicious behavior, such as Python's `exec()`. If such functions are identified within the source code, the analysis is immediately halted, and the file is flagged as unprocessable. This control mechanism is crucial for preventing infinite execution loops or system instability.

#### Dynamic Information Gathering

If the pre-execution checks are passed, the program proceeds to execution. The primary goal of this phase is to construct and analyze the quantum circuits as they are built at runtime.

To maintain a secure runtime environment, a further control is implemented. A monitoring mechanism tracks the call depth of functions during execution. If the call depth exceeds a predefined threshold, the execution is terminated to prevent infinite recursion, ensuring the analysis concludes in a timely manner.

As the code executes, all operations on the quantum circuit are meticulously recorded. This information is stored in a structured list, preserving the precise

sequence and order of operations. For each operation, the tool logs critical data, including its type (e.g., h, cx), the qubits it acts upon, and its exact location within the source code (line and column numbers). For operations with complex parameters, such as those for an rx gate, these specific values are also logged. This granular data collection allows for a deeper and more precise analysis.

### **Smells Detection**

Once the dynamic information gathering phase is complete, the collected data is passed to the specialized smell detectors. Each detector processes this comprehensive dataset to identify its specific type of smell based on a predefined set of criteria.

#### **3.2.4 Smell Detection Methodologies: Static Approach**

The tool employs a Static methodology for detecting quantum smells that are not immediately evident from a simple code scan and do not require the presence of a main execution block. This approach is particularly effective for analyzing code within functions, classes, and other modular components, where direct execution is not feasible. The term "static" is used to describe this method because the process simulates the behavior of the code rather than executing it in a traditional, live environment. Unlike the dynamic analysis, this method is designed to provide insights into how the code would behave without a complete, executable program.

Despite its name, this form of analysis does require a controlled execution of the code under examination. However, this is not a true runtime execution; rather, it's a simulated run-through to trace the flow of operations and build a representation of the quantum circuit. This allows the tool to accurately identify and analyze potential smells that are hidden within specific functions or classes, providing a comprehensive analysis that would be missed by relying solely on dynamic analysis. This unique blend of static and dynamic principles allows the tool to provide a complete and accurate report on the code's quality.

### Function-Based Code Analysis

The initial phase of this methodology involves systematically scanning the user-provided code to identify functions that manipulate quantum circuits. Specifically, the tool searches for functions that meet one of the following criteria:

- Functions that accept a `QuantumCircuit` object as an input parameter.
- Functions that return a `QuantumCircuit` object.
- Functions that instantiate a `QuantumCircuit` object within their scope.

### Executable Code Construction

Following the identification of relevant functions, the tool proceeds to construct an executable script for each function to facilitate a controlled simulation. This step is essential to bridge the gap between a standalone code snippet and a complete, runnable program, enabling the analysis to proceed without errors.

For each targeted function, a new file is dynamically generated. This file serves as a wrapper, containing a main execution block. The wrapper ensures that the function can be called and its behavior simulated. To prevent runtime errors, the tool automatically imports any necessary libraries and classes.

To handle dependencies of which the implementation is unknown, such as external classes or unimported functions, the tool employs mock objects. These mock objects simulate the behavior of the original classes and functions, allowing the simulation to proceed without requiring a full, operational environment. This mocking strategy ensures that the analysis remains focused on the quantum code itself, isolating its behavior from external dependencies and guaranteeing the execution's stability.

### The Auto-Fixing Mechanism

Following the generation of the executable script, a subsequent auto-fixing phase is initiated to handle any runtime errors that may arise during the simulation. This mechanism is crucial for ensuring that the analysis can proceed smoothly, as the generated scripts may contain unforeseen issues due to their dynamic construction.

The auto-fixing process involves executing the dynamically generated code and catching any exceptions. If a runtime error occurs, the system identifies the error type and dynamically injects additional code into the main execution block to resolve it. The logic for these fixes is predefined for a set of common, fundamental runtime errors. For instance, if an error indicates a missing variable or an uninitialized object, the system can automatically insert a declaration or instantiation to resolve the issue.

Given that not all possible error types have been tested and to prevent excessively long fixing loops, the number of auto-fix iterations is limited by a predefined threshold. This measure ensures that the process remains efficient and prevents the tool from getting stuck in an endless cycle of code modifications. The auto-fixing mechanism is a key component in making the source code simulation robust and reliable, allowing the tool to effectively analyze code that would otherwise be non-executable.

### **Dynamic Analysis and Smell Detection**

Following the successful auto-fixing and creation of a runnable script, the tool proceeds with the dynamic analysis phase. As detailed in Section 3.2.3, this methodology involves the direct execution of the code to gather comprehensive runtime information about the quantum circuit. It is crucial to note that this analysis is performed on the fixed code, which has been modified and debugged by the auto-fixing mechanism.

### **The Mapping Phase**

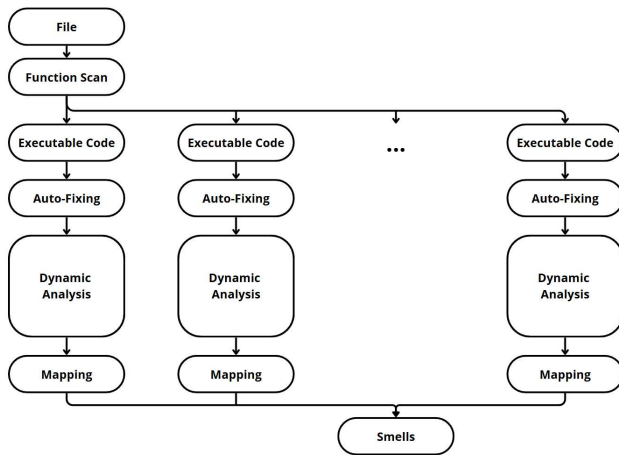
The final and critical step in this methodology is the mapping phase, which is essential for providing accurate feedback to the user. Since the analysis is performed on dynamically generated and potentially modified code, a mechanism is required to correlate the detected smells back to their original location in the user's source file. This is handled by a two-stage mapping process that exclusively tracks changes in line numbers.

The first stage of mapping is embedded within the auto-fixing mechanism. It maintains a running record of line number changes in the executable script. This ensures that any adjustments made during the auto-fix phase —such as adding a

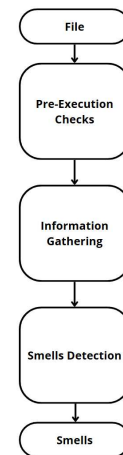
new variable declaration— do not invalidate the subsequent analysis.

The second stage of mapping creates a final correspondence between the lines of the generated main block and the original function being analyzed. This crucial step translates the location of a detected smell from the temporary, executable file back to the original source code, allowing the tool to report the issue with a precise line number.

It is important to note that this mapping focuses solely on line numbers, as column tracking is not maintained due to the dynamic nature of the code generation and fixing process. This approach guarantees that the tool’s reports are directly actionable for the user.



**Figure 3.3:** Static Smell Detection.



**Figure 3.4:** Dynamic Smell Detection.

### 3.2.5 Smell Explanation: An LLM-Powered Architecture

The tool’s functionality for explaining and resolving detected quantum smells is powered by a dedicated architecture centered around a Large Language Model (LLM). This component is designed to provide contextualized, human-readable feedback to the user. The system’s core is the Explainer class, which acts as a central hub and implements the Factory design pattern. This pattern dynamically instantiates the correct explainer subclass based on the specific smell type identified. Each subclass is responsible for crafting a precise and detailed prompt for the LLM. This prompt is dynamically generated using a combination of static information—including a general description of the smell and the analysis method (static or dynamic)—and

specific details derived from the detected smell instance, such as the exact location and relevant metrics.

The Explainer class then uses this prompt to interact with a commercial LLM via its API. This approach allows the tool to generate a tailored explanation and suggest specific resolutions for each unique smell instance, significantly enhancing the user experience by providing actionable insights beyond a simple detection report. The modularity of this design ensures that as new smell types are introduced, a corresponding explainer subclass can be easily added without modifying the core system logic.

### **LLM Integration via the OpenRouter API**

The tool leverages a Large Language Model (LLM) to generate detailed explanations and resolution suggestions for the detected quantum smells. This is achieved through a flexible API integration with OpenRouter, a platform that provides a unified interface for accessing various LLMs, including free and paid models. This modular design allows users to customize the LLM based on their preferences, model availability, and specific requirements.

The base model integrated in the tool is the DeepSeek R1 Distill Llama 70B (free), chosen for its robust performance and cost-effectiveness. This model offers an excellent balance between latency and throughput for a free-tier service. The latency of the provider is particularly low, ensuring a quick time-to-first-token, while its throughput remains competitive. This combination of speed and efficiency makes it well-suited for a wide range of use cases within the tool.

The user can configure the tool with their own OpenRouter API key and select a preferred model. The system is designed to be highly adaptable; by simply changing the model name in the configuration, users can seamlessly switch to other free or paid models available on the OpenRouter platform, ensuring the tool can evolve with the ever-changing landscape of LLMs.

## 3.3 Design Choices

The tool's user interface is designed to accommodate different user needs and workflows, providing two distinct access points: a Command-Line Interface (CLI) for advanced users and project-level analysis, and a Graphical User Interface (GUI) for a more intuitive, single-file experience.

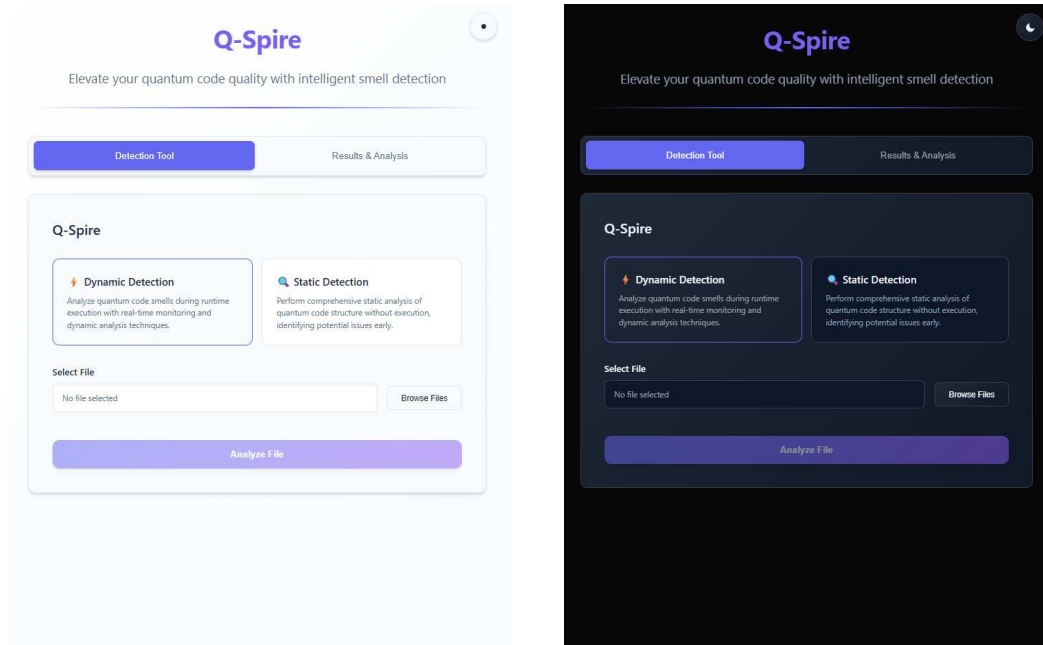
### 3.3.1 Command-Line Interface (CLI)

The Command-Line Interface provides a robust solution for streamlined and automated analysis, primarily focused on high-level detection rather than in-depth, location-specific detail. This interface is particularly suited for developers and researchers who need to integrate the tool into existing build systems or perform project-wide batch processing. To initiate an analysis, users must specify the absolute path to the quantum source file or project directory they wish to inspect. For individual files, the CLI produces a direct output to the console, listing the detected code smells. When analyzing entire projects, the CLI is configured to write a separate CSV file for each source file containing detected smells, storing all results within a designated output folder. This structured approach to data logging facilitates post-analysis review and integration with other reporting tools, providing a valuable mapping and analysis of the overall smell distribution across the project.

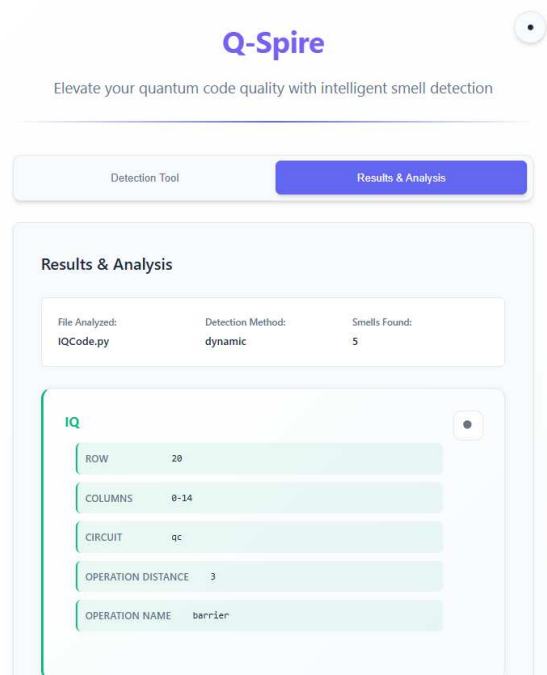
### 3.3.2 Graphical User Interface (GUI)

For a more user-friendly and direct interaction, the tool offers a comprehensive Graphical User Interface. The GUI is designed to simplify the detection process, particularly for new users or for quick, interactive analysis of single quantum files. The interface supports both static and dynamic detection methods, with the user simply selecting the target file for analysis. Upon completion, all detected smells are presented on the "Result and Analysis" screen. This dedicated view provides detailed information for each identified smell and features a button to query an integrated Large Language Model (LLM) for deeper explanations. To further enhance the user experience, the GUI also includes an on-screen code viewer that highlights the exact

location of each detected smell, allowing for immediate visual inspection and a clear understanding of the issue. A toggle for switching between a dark and a light theme is also available, ensuring visual comfort during extended usage.



**Figure 3.5:** The tool’s user interface is displayed in both light (left) and dark (right) themes, illustrating its adaptable visual presentation.



**Figure 3.6:** Smell Detection Instance on the file IQCode.py using dynamic analysis.

# Evaluation

The primary goal of this evaluation is to validate the tool's effectiveness and practical utility in a real-world context. To achieve this, a quantitative static analysis was conducted on a diverse dataset of open-source quantum computing projects. By analyzing a curated set of projects, this evaluation aims to provide empirical evidence of the tool's practical value for developers working with Qiskit framework, thereby supporting the broader objective of improving the quality and sustainability of quantum software.

## 4.1 Project Selection Criteria

The selection of a representative and reliable dataset was a crucial step in the evaluation process. A total of 40 open-source projects were meticulously chosen from GitHub, adhering to a strict set of criteria to ensure data integrity and relevance.

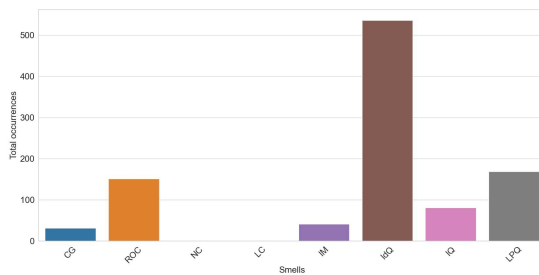
First, projects had to be publicly available via the GitHub API, written in Python, and using the Qiskit framework. To ensure maturity and active maintenance, a minimum of 50 commits over at least six months was required. This temporal filter excluded small, educational "toy" examples. Finally, a certain number of stars was required, serving as a proxy for community engagement and consistency. This helped to filter out unmaintained or non-substantial repositories, ensuring a realistic test environment.

**Table 4.1:** Quantum Code Smells detected in the analyzed projects

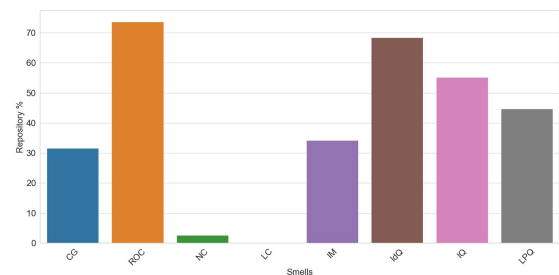
| Repo                          | CG | ROC | NC | LC | IM  | IdQ | IQ | LPQ |
|-------------------------------|----|-----|----|----|-----|-----|----|-----|
| isd-quantum                   | 0  | 1   | 0  | 0  | 1   | 3   | 1  | 0   |
| kaleidoscope                  | 0  | 0   | 0  | 0  | 0   | 0   | 0  | 0   |
| mpqf                          | 1  | 0   | 0  | 0  | 2   | 62  | 3  | 5   |
| piQture                       | 0  | 10  | 0  | 0  | 2   | 62  | 3  | 0   |
| planqk-quantum                | 0  | 8   | 0  | 0  | 0   | 16  | 5  | 0   |
| prototype-entanglement-forgin | 0  | 1   | 0  | 0  | 0   | 0   | 0  | 0   |
| prototype-qrao                | 0  | 4   | 0  | 0  | 0   | 5   | 1  | 0   |
| qBraid                        | 14 | 5   | 0  | 0  | 0   | 41  | 3  | 39  |
| qelm                          | 0  | 6   | 1  | 0  | 0   | 16  | 1  | 8   |
| QIRT                          | 1  | 0   | 0  | 0  | 0   | 0   | 0  | 0   |
| qisjob                        | 0  | 0   | 0  | 0  | 0   | 0   | 0  | 4   |
| qiskit-aakash                 | 10 | 62  | 0  | 0  | 158 | 825 | 89 | 44  |
| qiskit-addon-aqc-tensor       | 0  | 5   | 0  | 0  | 1   | 22  | 3  | 1   |
| qiskit-addon-cutting          | 1  | 14  | 0  | 0  | 7   | 56  | 6  | 0   |
| qiskit-addon-mpf              | 0  | 1   | 0  | 0  | 0   | 2   | 0  | 0   |
| qiskit-addon-mthree           | 0  | 25  | 5  | 0  | 5   | 129 | 7  | 9   |
| qiskit-addon-obp              | 0  | 0   | 0  | 0  | 0   | 6   | 0  | 0   |
| qiskit-addon-utils            | 0  | 7   | 0  | 0  | 0   | 31  | 1  | 3   |
| qiskit-algorithms             | 0  | 20  | 0  | 0  | 2   | 27  | 10 | 1   |
| qiskit-alice-bob-provider     | 0  | 7   | 0  | 0  | 8   | 16  | 4  | 30  |
| qiskit-braket-provider        | 0  | 1   | 0  | 0  | 6   | 64  | 5  | 5   |
| qiskit-classroom-converter    | 0  | 0   | 0  | 0  | 0   | 3   | 1  | 0   |
| qiskit-experiments            | 2  | 1   | 0  | 0  | 0   | 0   | 0  | 6   |
| qiskit-ibm-experiment         | 0  | 0   | 0  | 0  | 0   | 0   | 0  | 2   |
| qiskit-ibm-runtime            | 2  | 2   | 0  | 0  | 7   | 38  | 8  | 30  |
| qiskit-machine-learning       | 0  | 8   | 0  | 0  | 1   | 13  | 8  | 2   |
| qiskit-on-iqm                 | 0  | 4   | 0  | 0  | 1   | 5   | 0  | 17  |
| qiskit-pasqal-provider        | 3  | 0   | 0  | 0  | 0   | 0   | 0  | 0   |
| qiskit-serverless             | 0  | 6   | 0  | 0  | 0   | 6   | 0  | 0   |
| qiskit-zx-transpiler          | 1  | 0   | 0  | 0  | 1   | 58  | 3  | 0   |
| qonwaygameoflife              | 0  | 1   | 0  | 0  | 0   | 0   | 0  | 0   |
| qsolvers                      | 2  | 5   | 0  | 0  | 0   | 5   | 4  | 0   |
| quantpiler-v0                 | 0  | 5   | 0  | 0  | 3   | 13  | 5  | 0   |
| quantum-nodes                 | 1  | 0   | 0  | 0  | 0   | 0   | 0  | 0   |
| quantum-pipeline              | 0  | 1   | 0  | 0  | 0   | 0   | 0  | 0   |
| quantum-solver                | 1  | 19  | 0  | 0  | 1   | 11  | 5  | 2   |
| qurrium                       | 2  | 1   | 0  | 0  | 0   | 0   | 0  | 4   |
| qutes-lang                    | 0  | 2   | 0  | 0  | 0   | 0   | 0  | 0   |
| rivet                         | 0  | 5   | 0  | 0  | 0   | 3   | 1  | 10  |
| xtraorbitals.xyz              | 0  | 1   | 0  | 0  | 0   | 1   | 0  | 0   |

## 4.2 Graphs Analysis

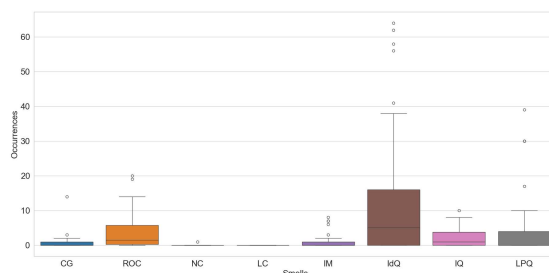
To more effectively visualize the distribution of code smells and identify trends within the dataset, a series graphs were generated. To ensure the significance and readability of the results, it was necessary to remove two projects, qiskit-addon-mthree and qiskit-aakash, from the calculation of statistics and the visualization. These projects were identified as outliers due to their excessively high number of code smells, which, if included, would have distorted the scale of the charts and made it difficult to interpret the data for the other projects in the sample. Therefore, the graphical analysis is based on a refined version of the dataset, which reflects a more representative distribution of the majority of the analyzed projects.



**Figure 4.1:** Smells Total Occurences



**Figure 4.2:** Repository Smells Percentage



**Figure 4.3:** Smells Boxplot



**Figure 4.4:** Smells Correlation Matrix

## **Barcharts**

The 'Total occurrences' graph in 4.1 clearly indicates that IdQ is the most frequently detected code smell, accounting for the highest absolute number of instances. This finding suggests that practices leading to the IdQ smell are the most common source of code defects in a quantitative sense. In contrast, the 'Repository Smells Percentage' graph in 4.2 reveals that ROC is the most prevalent code smell, as it was detected in a higher percentage of repositories than any other type. This distinction is crucial: while IdQ appears in larger quantities, ROC is more widely distributed across the projects, highlighting a different kind of challenge for maintainability.

Furthermore, the low incidence of smells like NC and LC suggests that these specific code patterns are less common in the Qiskit ecosystem.

## **Boxplot**

The box plot in 4.3 provides a concise summary of code smell distribution, showing a significant difference between total counts and per-project presence. For most smells, like CG, NC, and LC, the median occurrence is near zero, indicating their general infrequency within individual projects. Conversely, the IdQ box is taller and higher on the y-axis, confirming its substantial presence and wide spread across repositories. The outliers for IdQ, ROC, and IQ suggest a few projects have a disproportionately high number of defects, likely representing particularly problematic repositories.

## **Correlation Matrix**

The correlation matrix in 4.4 reveals important interdependencies among code smells. Strong positive correlations were found between IM, IdQ, and IQ, with coefficients of 0.60, 0.62, and 0.58. This indicates that projects with a high number of one of these smells often have a high number of the others, suggesting a common underlying cause. Moderate correlations were also observed between ROC and IQ (0.67) and CG and LPQ (0.62). These links suggest that certain code patterns are associated with other forms of code decay. Conversely, the low correlation for NC and LC reinforces their low frequency and independent nature in the dataset.

# Conclutions

## 5.1 Main Conclusions

The primary objective of this thesis was to develop and validate a practical tool for detecting quantum code smells. Building upon the foundational work of Chen et al., who formally defined a set of quantum code smells, a new tool, Q-Spire, was implemented to overcome the limitations of previous detection methods. A key innovation of this tool is its capability to perform both static and dynamic analysis, providing a more comprehensive view of code quality. The extensive evaluation conducted on real-world, open-source projects demonstrated that Q-Spire is a robust and effective solution, successfully identifying code smells that can hinder maintainability and readability. The results underscore that despite its nascent stage, the quantum computing community has significant room for improvement in software engineering practices, highlighting the critical need for tools like Q-Spire to foster the development of more reliable and sustainable quantum software.

The Q-Spire tool is available at the following GitHub repository:  
<https://github.com/Riccardoalfieri2003/Q-Spire>

## 5.2 Future Works

The Q-Spire tool, while demonstrating its effectiveness and practicality, remains in a foundational stage and presents several clear avenues for future research and development.

Addressing these limitations will be crucial for the tool’s maturation and broader adoption within the quantum computing community.

First, a critical area for improvement lies in enhancing the autofix mechanism integrated within the static detection phase. Currently, the autofix functionality has been validated only against a limited set of recurring runtime errors. As a result, the tool may fail to apply corrective actions when encountering less frequent or more complex patterns, potentially overlooking code segments that still exhibit smell characteristics. Expanding the autofix capabilities to cover a broader spectrum of errors would significantly improve the robustness and applicability of the tool.

Another area for improvement is the mapping process, which supports the autofix phase. Although conceptually straightforward, this process is currently inefficient and can become a bottleneck when analyzing large files, requiring several minutes to complete. Optimizing the mapping algorithm would significantly enhance the tool’s scalability and responsiveness.

Another topic to consider is the validation of LLM-generated explanations, which is a significant limitation. The current implementation relies on a single in-context learning approach without a formal mechanism to evaluate the sufficiency or accuracy of the explanations provided. Future research should explore methods to validate these responses, perhaps through expert human assessments or by integrating more advanced prompt engineering techniques to ensure the explanations are both clear and exhaustive.

Lastly, a promising direction for future work involves the User Interface (UI) validation. The current work did not include a formal usability study, such as surveys or user testing, to assess the user-friendliness and intuitiveness of the GUI. Future work should focus on gathering quantitative and qualitative feedback from developers to refine the interface, making the tool more accessible and user-friendly for a diverse range of users.

# Bibliography

- [1] E. Tom, A. Aurum, and R. Vidgen, “An exploration of technical debt,” *Journal of Systems and Software*, vol. 86, no. 6, pp. 1498–1516, 2013. (Cited on page 1)
- [2] N. S. Alves, T. S. Mendes, M. G. De Mendonça, R. O. Spínola, F. Shull, and C. Seaman, “Identification and management of technical debt: A systematic mapping study,” *Information and Software Technology*, vol. 70, pp. 100–121, 2016. (Cited on page 1)
- [3] W. N. Behutiye, P. Rodríguez, M. Oivo, and A. Tosun, “Analyzing the concept of technical debt in the context of agile software development: A systematic literature review,” *Information and Software Technology*, vol. 82, pp. 139–158, 2017. (Cited on page 1)
- [4] N. Zazworka, R. O. Spínola, A. Vetro', F. Shull, and C. Seaman, “A case study on effectively identifying technical debt,” in *Proceedings of the 17th International Conference on Evaluation and Assessment in Software Engineering*, 2013, pp. 42–47. (Cited on page 1)
- [5] D. Das, A. A. Maruf, R. Islam, N. Lambaria, S. Kim, A. S. Abdelfattah, T. Cerny, K. Frajtak, M. Bures, and P. Tisnovsky, “Technical debt resulting from architectural degradation and code smells: a systematic mapping study,” *ACM SIGAPP Applied Computing Review*, vol. 21, no. 4, pp. 20–36, 2022. (Cited on page 1)
- [6] S. Wehaibi, E. Shihab, and L. Guerrouj, “Examining the impact of self-admitted technical debt on software quality,” in *2016 IEEE 23Rd international conference on*

- software analysis, evolution, and reengineering (SANER)*, vol. 1. IEEE, 2016, pp. 179–188. (Cited on page 1)
- [7] Y. Guo, R. O. Spínola, and C. Seaman, “Exploring the costs of technical debt management—a case study,” *Empirical Software Engineering*, vol. 21, no. 1, pp. 159–182, 2016. (Cited on page 1)
- [8] M. Piattini, G. Peterssen, and R. Pérez-Castillo, “Quantum computing: A new software engineering golden age,” *ACM SIGSOFT Software Engineering Notes*, vol. 45, no. 3, pp. 12–14, 2021. (Cited on page 1)
- [9] M. Horowitz and E. Grumbling, “Quantum computing: progress and prospects,” 2019. (Cited on pages 1 e 4)
- [10] H. Bhatt and S. Gautam, “Quantum computing: A new era of computer science,” in *2019 6th International Conference on Computing for Sustainable Global Development (INDIACom)*. IEEE, 2019, pp. 558–561. (Cited on page 1)
- [11] M. Openja, M. M. Morovati, L. An, F. Khomh, and M. Abidi, “Technical debts and faults in open-source quantum software systems: An empirical study,” *Journal of Systems and Software*, vol. 193, p. 111458, 2022. (Cited on page 1)
- [12] Y. Ishimoto, Y. Nakamura, R. Katsube, N. Sato, H. Ogawa, M. Kondo, Y. Kamei, and N. Ubayashi, “An empirical study on self-admitted technical debt in quantum software,” in *2024 31st Asia-Pacific Software Engineering Conference (APSEC)*. IEEE, 2024, pp. 41–50. (Cited on page 1)
- [13] Q. Chen, R. Câmara, J. Campos, A. Souto, and I. Ahmed, “The smelly eight: An empirical study on the prevalence of code smells in quantum computing,” in *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*. IEEE, 2023, pp. 358–370. (Cited on pages 2 e 6)
- [14] N. Wadhwa, J. Pradhan, A. Sonwane, S. P. Sahu, N. Natarajan, A. Kanade, S. Parthasarathy, and S. Rajamani, “Core: Resolving code quality issues using llms,” *Proceedings of the ACM on Software Engineering*, vol. 1, no. FSE, pp. 789–811, 2024. (Cited on page 2)

- [15] D. Nam, A. Macvean, V. Hellendoorn, B. Vasilescu, and B. Myers, "Using an llm to help with code understanding," in *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*, 2024, pp. 1–13. (Cited on page 2)
- [16] A. Steane, "Quantum computing," *Reports on Progress in Physics*, vol. 61, no. 2, p. 117, 1998. (Cited on page 3)
- [17] D. Aharonov, "Quantum computation," *Annual Reviews of Computational Physics VI*, pp. 259–346, 1999. (Cited on page 3)
- [18] A. Datta, A. Shaji, and C. M. Caves, "Quantum discord and the power of one qubit," *Physical review letters*, vol. 100, no. 5, p. 050502, 2008. (Cited on page 3)
- [19] M. J. Renner and Č. Brukner, "Computational advantage from a quantum superposition of qubit gate orders," *Physical Review Letters*, vol. 128, no. 23, p. 230503, 2022. (Cited on page 3)
- [20] R. Jozsa and N. Linden, "On the role of entanglement in quantum-computational speed-up," *Proceedings of the Royal Society of London. Series A: Mathematical, Physical and Engineering Sciences*, vol. 459, no. 2036, pp. 2011–2032, 2003. (Cited on page 3)
- [21] M. Hirvensalo, "Interference as a computational resource: a tutorial," *Natural Computing*, vol. 17, no. 1, pp. 201–219, 2018. (Cited on page 3)
- [22] F. Arute, K. Arya, R. Babbush, D. Bacon, J. C. Bardin, R. Barends, R. Biswas, S. Boixo, F. G. Brandao, D. A. Buell *et al.*, "Quantum supremacy using a programmable superconducting processor," *Nature*, vol. 574, no. 7779, pp. 505–510, 2019. (Cited on page 3)
- [23] I. L. Markov, A. Fatima, S. V. Isakov, and S. Boixo, "Quantum supremacy is both closer and farther than it appears," *arXiv preprint arXiv:1807.10749*, 2018. (Cited on page 3)
- [24] B. M. Terhal, "Quantum supremacy, here we come," *Nature Physics*, vol. 14, no. 6, pp. 530–531, 2018. (Cited on page 3)

- [25] Y. Alexeev, D. Bacon, K. R. Brown, R. Calderbank, L. D. Carr, F. T. Chong, B. DeMarco, D. Englund, E. Farhi, B. Fefferman *et al.*, “Quantum computer systems for scientific discovery,” *PRX quantum*, vol. 2, no. 1, p. 017001, 2021. (Cited on page 4)
- [26] J. Biamonte, P. Wittek, N. Pancotti, P. Rebentrost, N. Wiebe, and S. Lloyd, “Quantum machine learning,” *Nature*, vol. 549, no. 7671, pp. 195–202, 2017. (Cited on page 4)
- [27] R. Orús, S. Muel, and E. Lizaso, “Quantum computing for finance: Overview and prospects,” *Reviews in Physics*, vol. 4, p. 100028, 2019. (Cited on page 4)
- [28] N. Gisin, G. Ribordy, W. Tittel, and H. Zbinden, “Quantum cryptography,” *Reviews of modern physics*, vol. 74, no. 1, p. 145, 2002. (Cited on page 4)
- [29] M. Möller and C. Vuik, “On the impact of quantum computing technology on future developments in high-performance scientific computing,” *Ethics and information technology*, vol. 19, no. 4, pp. 253–269, 2017. (Cited on page 4)
- [30] S. Aaronson, “The limits of quantum,” *Scientific American*, vol. 298, no. 3, pp. 62–69, 2008. (Cited on page 4)
- [31] L. Fortnow and J. Rogers, “Complexity limitations on quantum computation,” *Journal of Computer and System Sciences*, vol. 59, no. 2, pp. 240–252, 1999. (Cited on page 4)
- [32] S. Bravyi, O. Dial, J. M. Gambetta, D. Gil, and Z. Nazario, “The future of quantum computing with superconducting qubits,” *Journal of Applied Physics*, vol. 132, no. 16, 2022. (Cited on page 4)
- [33] H. B. Patel, S. Mishra, R. Jain, and N. Kansara, “The future of quantum computing and its potential applications,” *Journal For Basic Sciences (forthcoming). academia. edu*, 2023. (Cited on page 4)
- [34] V. Hassija, V. Chamola, A. Goyal, S. S. Kanhere, and N. Guizani, “Forthcoming applications of quantum computing: peeking into the future,” *IET Quantum Communication*, vol. 1, no. 2, pp. 35–41, 2020. (Cited on page 4)

- 
- [35] A. Peres and D. R. Terno, "Hybrid classical-quantum dynamics," *Physical Review A*, vol. 63, no. 2, p. 022101, 2001. (Cited on page 4)
- [36] M. Suchara, Y. Alexeev, F. Chong, H. Finkel, H. Hoffmann, J. Larson, J. Osborn, and G. Smith, "Hybrid quantum-classical computing architectures," in *Proceedings of the 3rd International Workshop on Post-Moore Era Supercomputing, 2018.*, 2018. (Cited on page 4)
- [37] M. Tufano, F. Palomba, G. Bavota, R. Oliveto, M. Di Penta, A. De Lucia, and D. Poshyvanyk, "When and why your code starts to smell bad," in *2015 IEEE/ACM 37th IEEE International Conference on Software Engineering*, vol. 1. IEEE, 2015, pp. 403–414. (Cited on page 4)
- [38] J. A. M. Santos, J. B. Rocha-Junior, L. C. L. Prates, R. S. Do Nascimento, M. F. Freitas, and M. G. De Mendonça, "A systematic review on the code smell effect," *Journal of Systems and Software*, vol. 144, pp. 450–477, 2018. (Cited on page 4)
- [39] E. Iannone, R. Guadagni, F. Ferrucci, A. De Lucia, and F. Palomba, "The secret life of software vulnerabilities: A large-scale empirical study," *IEEE Transactions on Software Engineering*, 2022. (Cited on page 4)
- [40] S. Lambiase, G. Catolino, D. A. Tamburri, A. Serebrenik, F. Palomba, and F. Ferrucci, "Good fences make good neighbours? on the impact of cultural and geographical dispersion on community smells," 2022. (Cited on page 4)
- [41] M. Piattini, M. Serrano, R. Perez-Castillo, G. Petersen, and J. L. Hevia, "Toward a quantum software engineering," *IT Professional*, vol. 23, no. 1, pp. 62–66, 2021. (Cited on page 5)
- [42] A. Conlon, D. Pellegrino, J. K. Slingerland, S. Dooley, and G. Kells, "Error generation and propagation in majorana-based topological qubits," *Physical Review B*, vol. 100, no. 13, p. 134307, 2019. (Cited on page 5)
- [43] M. I. Jordan and T. M. Mitchell, "Machine learning: Trends, perspectives, and prospects," *Science*, vol. 349, no. 6245, pp. 255–260, 2015. (Cited on page 5)

- 
- [44] Y. LeCun, Y. Bengio, and G. Hinton, "Deep learning," *nature*, vol. 521, no. 7553, pp. 436–444, 2015. (Cited on page 5)
- [45] S. Feuerriegel, J. Hartmann, C. Janiesch, and P. Zschech, "Generative ai," *Business & Information Systems Engineering*, vol. 66, no. 1, pp. 111–126, 2024. (Cited on page 5)
- [46] C. Y. Kim, C. P. Lee, and B. Mutlu, "Understanding large-language model (llm)-powered human-robot interaction," in *Proceedings of the 2024 ACM/IEEE international conference on human-robot interaction*, 2024, pp. 371–380. (Cited on page 5)
- [47] N. Wies, Y. Levine, and A. Shashua, "The learnability of in-context learning," *Advances in Neural Information Processing Systems*, vol. 36, pp. 36 637–36 651, 2023. (Cited on page 5)
- [48] Z. Chu, S. Wang, J. Xie, T. Zhu, Y. Yan, J. Ye, A. Zhong, X. Hu, J. Liang, P. S. Yu *et al.*, "Llm agents for education: Advances and applications," *arXiv preprint arXiv:2503.11733*, 2025. (Cited on page 5)
- [49] J.-Y. Yao, K.-P. Ning, Z.-H. Liu, M.-N. Ning, Y.-Y. Liu, and L. Yuan, "Llm lies: Hallucinations are not bugs, but features as adversarial examples," *arXiv preprint arXiv:2310.01469*, 2023. (Cited on page 5)
- [50] S. Dai, C. Xu, S. Xu, L. Pang, Z. Dong, and J. Xu, "Bias and unfairness in information retrieval systems: New challenges in the llm era," in *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, 2024, pp. 6437–6447. (Cited on page 5)
- [51] A. Javadi-Abhari, M. Treinish, K. Krsulich, C. J. Wood, J. Lishman, J. Gacon, S. Martiel, P. D. Nation, L. S. Bishop, A. W. Cross *et al.*, "Quantum computing with qiskit," *arXiv preprint arXiv:2405.08810*, 2024. (Cited on page 6)
- [52] D. Puzzuoli, C. J. Wood, D. J. Egger, B. Rosand, and K. Ueda, "Qiskit dynamics: A python package for simulating the time dynamics of quantum systems," *Journal of Open Source Software*, vol. 8, no. 90, p. 5853, 2023. (Cited on page 6)

- [53] M. Ben-Ari, "Constructivism in computer science education," *Journal of computers in Mathematics and Science Teaching*, vol. 20, no. 1, pp. 45–73, 2001. (Cited on page 6)
- [54] A. B. Tucker, "Strategic directions in computer science education," *ACM Computing Surveys (CSUR)*, vol. 28, no. 4, pp. 836–845, 1996. (Cited on page 6)

# Ringraziamenti

Non vorrei dilungarmi molto sui ringraziamenti siccome non sono bravo con le parole.

Un ringraziamento speciale va ai miei amici stretti, presenti e non, che, nonostante le mie stranezze, mi hanno sopportato nel corso degli anni e mi hanno fatto crescere.

Vorrei inoltre ringraziare il professore Stefano Lambiase, che adesso è un vero professore. Anche se non è più a Salerno, mi è stato accanto negli ultimi mesi.

Un ultimo ringraziamento va alla famiglia, specialmente ai miei genitori e mio fratello Alessandro, con i quali condivido le memorie più belle che ho. Devo specialmente ringraziare i miei genitori che ora dovranno supportarmi anche economicamente.

Mamma, mi raccomando, non piangere quando non ci sono.

Grazie a tutti, Mahalo.